FINAL PROJECT DEFENSE

DevOps Health Monitoring Dashboard

my-devops-project

Author
Vitalii Zaburdaiev

Course
DevOpsUA6

Date
April 2026

github.com/zaburdaev/my-devops-project

oskalibriya/health-dashboard

Обзор проекта

Flask-приложение для мониторинга здоровья системы

Демонстрирует полный цикл DevOps: от кода до production-ready инфраструктуры.

CPU / RAM / Disk | REST API | Metrics & Logs

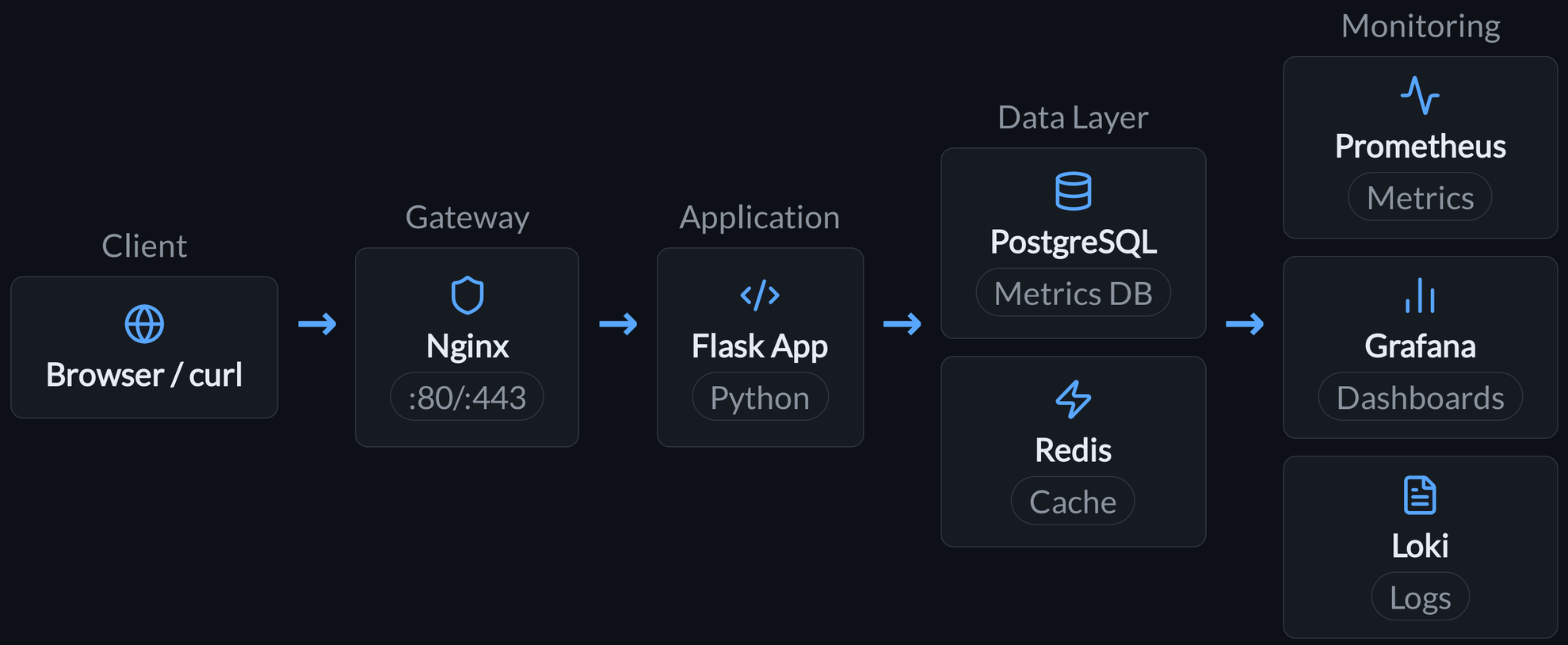
GitHub Repository
github.com/zaburdaev/my-devops-project

Docker Hub
oskalibriya/health-dashboard

Цель проекта
Демонстрация DevOps практик на реальном приложении

7	12	3
Services	Unit Tests	API Endpoints

Архитектура решения



7 сервисов управляются через Docker Compose

Технологический стек

Python / Flask
Web Application

Docker
Containerization

Kubernetes
Orchestration

Terraform
Infrastructure as Code

Ansible
Automation

Prometheus
Metrics

Grafana
Dashboards

GitHub Actions
CI/CD Pipeline

Приложение (Flask)

API Endpoints

- GET /health — Статус сервисов приложения
- GET /metrics — Prometheus-совместимые метрики
- GET /api/system-info — CPU, RAM, Disk (JSON)

Мониторинг системы

- CPU Usage
- Memory Usage
- Disk Usage
- JSON Logging → Loki

PostgreSQL
Хранение метрик

Redis
Кэширование

12 unit тестов (pytest)
All Passing

Контейнеризация (Docker)

Multi-Stage Dockerfile

1 Stage 1: Builder
python:3.11-slim — install deps

2 Stage 2: Runtime
Копируем только нужные артефакты

Result
Минимальный образ, без build-зависимостей

Размер образа уменьшен на ~60%

Docker Compose — 7 сервисов

- **app** — Flask Application
- **postgres** — Database
- **redis** — Cache
- **nginx** — Reverse Proxy
- **prometheus** — Metrics Collector
- **grafana** — Visualization
- **loki** — Log Aggregation

Non-root user | Health checks | Secrets via env

Infrastructure as Code (Terraform)

Terraform Workflow

terraform init
Инициализация провайдера AWS

terraform plan
Предпросмотр изменений

terraform apply
Создание ресурсов в AWS

terraform destroy
Удаление инфраструктуры

AWS Resources AWS

EC2 Instance Compute
t2.micro, Ubuntu 22.04

Security Group Network
Ports: 22, 80, 443, 3000, 9090

Key Pair Security
SSH доступ

Идемпотентность: повторный apply не создаёт дубли

CI/CD Pipeline (GitHub Actions)

Code Push → Run Tests → Build Image → Push to Hub → Deploy

Test

- pytest — 12 unit тестов
- Coverage report
- Автоматический запуск на push

Build

- docker build (multi-stage)
- Тег: latest + git SHA
- Push → Docker Hub

Deploy

- SSH deploy на EC2
- docker compose up -d
- Автоматический health check

Kubernetes + Ansible

Kubernetes
Оркестрация контейнеров

Deployment
2 реплики, rolling update, liveness/readiness probes

Service
ClusterIP для внутренней маршрутизации

Helm Chart Bonus
Параметризованный chart, values.yaml

Ansible
Автоматизация настройки

Playbook
Установка Docker, настройка окружения, деплой

Inventory
Динамический инвентарь для AWS EC2

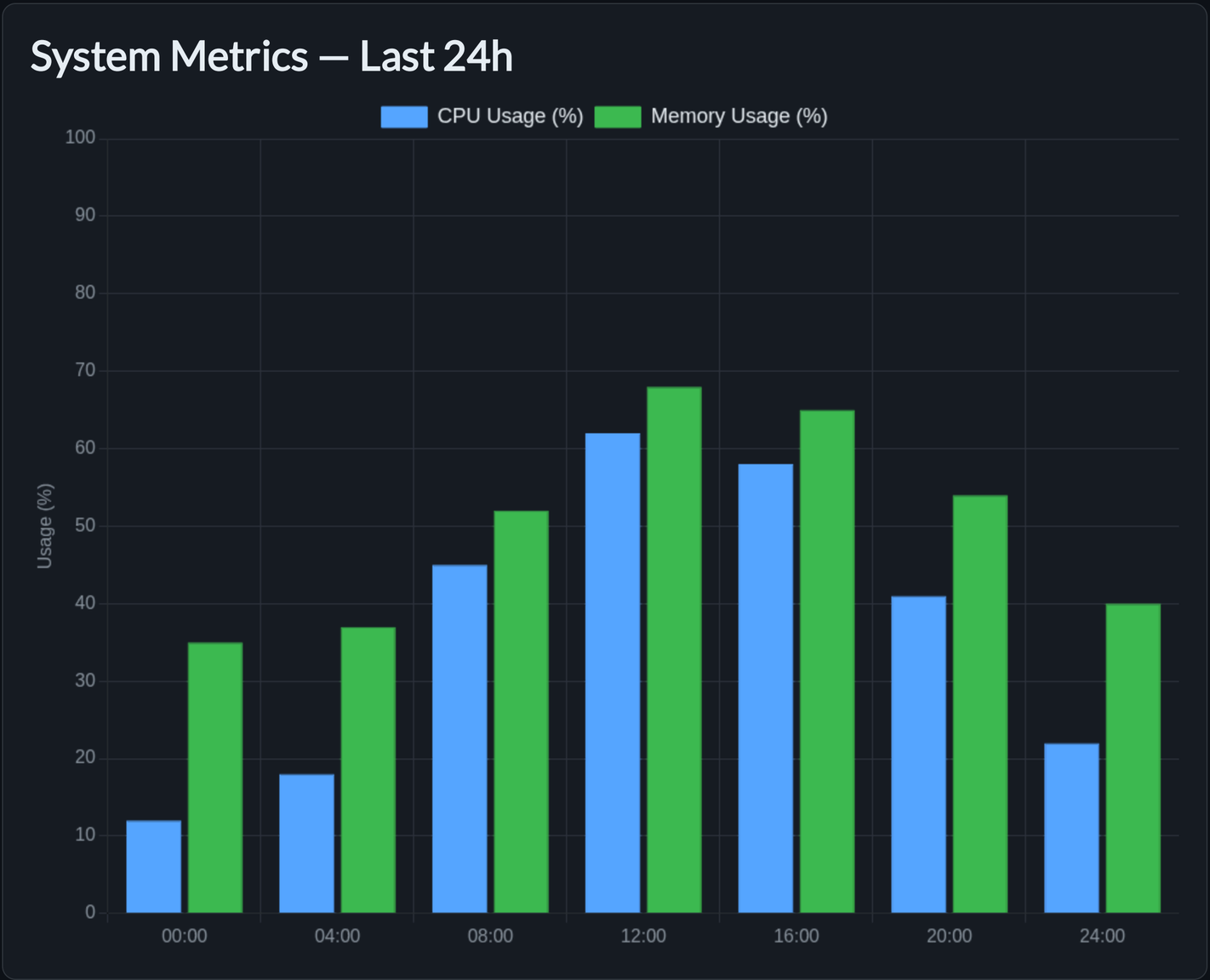
Idempotency
Безопасный повторный запуск без побочных эффектов

Мониторинг и логирование

Prometheus
Сбор метрик
Scrapes /metrics каждые 15 сек

Grafana
Визуализация
Дашборды CPU, Memory, HTTP

Loki
Агрегация логов
JSON логи из Flask → Loki → Grafana



Результаты и достижения

Критерий	Баллы	Статус
Собственное приложение	30 / 30	✓
Terraform	30 / 30	✓
Docker	30 / 30	✓
CI/CD	30 / 30	✓
Kubernetes	20 / 20	✓
Ansible	20 / 20	✓
Мониторинг	20 / 20	✓
Логирование	20 / 20	✓
Безопасность	10 / 10	✓
Тесты	10 / 10	✓
Документация	30 / 30	✓
Бонусы (Helm, multi-stage)	20 / 20	✓

240 / 240

Итоговый балл

Максимальный результат

- Полная автоматизация CI/CD
- Production-ready security
- Helm chart (бонус)
- Multi-stage Docker (бонус)

Итоги проекта: что реализовано

✅ Приложение и API

Flask dashboard, health endpoint, Prometheus metrics, unit-тесты.

✅ Контейнеризация

Multi-stage Dockerfile, docker-compose стек, non-root runtime, healthcheck.

✅ Автоматизация

GitHub Actions CI/CD, Terraform + Ansible, Kubernetes манифесты и Helm chart.

📊 Observability

Prometheus + Grafana с auto-provisioning dashboard и datasource.

🔐 Security hardening

Убраны дефолтные секреты, усилены примеры .env, audit и best practices в документации.

📚 Documentation

README/README_RU, beginner guide, demo script, troubleshooting и recovery runbook.

Безопасность и качество (финальная версия)

Секреты и конфигурация

- SECRET_KEY генерируется безопасно, если не задан env.
- Удалены слабые дефолтные креды из .env.example, K8s и Helm values.
- Grafana параметры вынесены в переменные окружения.

Код и зависимости

- Обновлены уязвимые Python зависимости.
- 12 unit-тестов стабильно проходят в CI.
- Nginx и Docker конфигурации усилены для production-сценария.

.gitignore hardening

Исключены state-файлы, ключи, отчёты аудита и локальные env.

Terraform security

SSH CIDR вынесен в переменную: удобно и безопасно ограничивать доступ.

SECURITY_AUDIT.md

Зафиксировано отсутствие реальных credentials в репозитории.

Документация

Обновлены README, CI/CD guide, troubleshooting и recovery разделы.

Устойчивость инфраструктуры и Recovery

Что добавлено в финальной версии

- Static Elastic IP для стабильного публичного доступа.
- Recovery runbook в AWS_DEPLOYMENT_RU.md и TROUBLESHOOTING_RU.md.
- Manual workflow infrastructure-recovery.yml для быстрого восстановления.
- Ansible deploy как безопасный fallback при сбое SSH deploy в CI/CD.

Recovery flow (упрощённо)

1. terraform apply
2. output IP/SSH
3. update secrets
4. deploy + verify

GitHub Actions

CI/CD + отдельный recovery workflow для аварийного восстановления.

DEPLOYMENT_SUMMARY.md

Единая точка правды по URL сервисов, проверкам и актуальному статусу деплоя.

Наблюдаемость после recovery

Проверка /health, Grafana dashboards и Prometheus targets после каждого восстановления.

ФИНАЛЬНАЯ ЗАЩИТА ПРОЕКТА

Спасибо за внимание!

Вопросы и ответы

Автор
Vitalii Zaburdaiev

Курс
DevOpsUA6

GitHub
github.com/zaburdaev/my-devops-project

Health Monitoring Dashboard · Final 2026